

AgentFlow — Traitement des documents PDF

Parsing, pipeline d'indexation, stratégies de chunking, outils et paramètres

Rapport d'avancement — plateforme RAG d'entreprise

1. Le pipeline complet d'un PDF

Chaque document PDF traverse cinq étapes. Chaque étape conserve les métadonnées de la précédente (pages, sections), ce qui permet des citations précises dans le chat.

- **UPLOAD** — le fichier est enregistré (uploads/<espace>/), statut LOADED.
- **PARSING** — Docling analyse la mise en page et produit des éléments typés (titres, paragraphes, tableaux, images), statut EXTRACTED.
- **CHUNKING** — la stratégie choisie découpe les éléments en chunks, avec métadonnées (section §, hash, nombre de tokens, langue).
- **EMBEDDING** — le modèle d'embedding de l'espace transforme chaque chunk en vecteur, à sa dimension NATIVE (384/768/1024/1536/3072).
- **INDEXATION** — texte + métadonnées dans PostgreSQL (table chunks), vecteur dans la table de dimension chunk_vectors_<dim> avec index HNSW. Statut INDEXED.

Re-traiter un document supprime ses anciens chunks et vecteurs (suppression en cascade) puis reconstruit tout : l'opération est répétable sans risque.

2. Le parsing PDF — comment ça marche

Outil : Docling (IBM, open source) — modèles d'analyse de mise en page, pas une simple extraction de texte.

Le parsing lit chaque page VISUELLEMENT et reconnaît le rôle de chaque zone :

- Titres (headings) — lignes numérotées (« 3.2 Congés ») ou courtes en majuscules ; ils construisent le plan du document.
- Paragraphes — le texte courant, dans l'ordre de lecture.
- Tableaux — cellules reconstruites avec leurs colonnes ; un tableau coupé sur deux pages est FUSIONNÉ en un seul tableau logique.
- Images — sauvegardées sur disque + une description générée par IA (vision), pour que l'image soit retrouvable par son sens et affichable dans le chat.
- Chaque élément garde sa PAGE et son ordre de lecture.

Points à vérifier dans la vue « Extracted » avant de chunker : les titres détectés sont-ils de vrais titres ? Les tableaux sont-ils complets ? Les numéros de page corrects ? La qualité du parsing plafonne tout le reste : un titre manqué ruine la stratégie « Section/Heading », un tableau cassé ruine les réponses tabulaires.

3. Le chunking — règles communes à toutes les stratégies

Les stratégies ne reçoivent jamais le PDF brut : elles parcourent les éléments parsés, dans l'ordre. Des règles structurales sont garanties AVANT toute stratégie :

- Un tableau n'est jamais coupé (chunk [TABLE] entier).
- Une image = son propre chunk (seulement si elle a une vraie description).
- Un titre ne devient jamais un chunk isolé : il devient le fil d'Ariane § de la section.
- Chaque chunk enregistre ses métadonnées : section §, hash md5 (anti-doublons), nombre de tokens

(tiktoken), langue détectée (fr/en).

4. Les 11 stratégies pour PDF — fonctionnement, outil, paramètres

4.1 Recursive (défaut recommandé)

Découpe au niveau le plus naturel disponible : d'abord entre paragraphes, puis entre phrases, jamais au milieu d'un mot. Un nouveau titre ouvre toujours un nouveau chunk (les sections ne se mélangent pas) et le fil d'Ariane § est préfixé.

Outil : [LlamaIndex SentenceSplitter](#) (découpe consciente des phrases).

Paramètre	Défaut	Plage	Explication
Chunk size	512	64-4000	Taille cible d'un chunk. PETIT (256-512) = recherche précise mais moins de contexte par résultat. GRAND (1000-1500) = plus de contexte mais résultats plus bruités.
Overlap	50	0-1000	Caractères répétés entre deux chunks consécutifs pour qu'une phrase à la frontière garde son contexte. Règle : environ 10% de la taille.

Quand l'utiliser : le choix par défaut pour du texte courant.

4.2 Fixed-size

Fenêtres uniformes de N caractères exactement, qui coupent à travers titres, phrases et tout le reste. C'est la BASELINE : volontairement brutale, elle sert de point de comparaison.

Outil : [LlamaIndex TokenTextSplitter](#).

Paramètre	Défaut	Plage	Explication
Chunk size	512	64-4000	Largeur fixe de la fenêtre.
Overlap	50	0-1000	Chevauchement entre fenêtres.

Quand l'utiliser : benchmarks et comparaisons ; rarement en production.

4.3 Sentence

Regroupe des phrases ENTières jusqu'à la limite — ne casse jamais une phrase.

Outil : [LlamaIndex SentenceSplitter \(mode fin\)](#).

Paramètre	Défaut	Plage	Explication
Max size	800	128-6000	Limite de remplissage d'un chunk en caractères.
Sentence overlap	1	0-5	Répète les N dernières phrases au début du chunk suivant.

Quand l'utiliser : textes denses où couper une phrase détruirait le sens (articles, livres).

4.4 Paragraph

Le paragraphe est l'unité de base : paragraphes entiers regroupés jusqu'à la limite, les petits fusionnés, un paragraphe géant est re-découpé en secours.

Outil : [Logique interne \(packing de paragraphes\)](#).

Paramètre	Défaut	Plage	Explication
Max size	1000	128-6000	Limite de remplissage ; un paragraphe seul plus grand est re-découpé.

Quand l'utiliser : documents bien rédigés aux paragraphes porteurs de sens (DOCX, articles).

4.5 Section / Heading

Tout le TEXTE sous un titre = un chunk (titre préfixé). Les tableaux et images de la section deviennent des chunks SÉPARÉS (avec le même §) : un tableau reste interrogeable indépendamment, sans diluer son embedding.

Outil : Logique interne, s'appuie sur les titres détectés par Docling.

Paramètre	Défaut	Plage	Explication
Max size	1500	128-6000	Une section plus grande est re-découpée, le titre re-préfixé sur chaque morceau pour qu'aucun fragment ne perde son identité.

Quand l'utiliser : documents à plan propre (contrats, politiques RH, manuels) et questions « valeur dans un tableau ».

4.6 Page

Une page = un chunk, tel quel. Aucune intelligence, mais totalement prévisible.

Outil : Logique interne (groupement par page).

Aucun paramètre.

Quand l'utiliser : documents où la page est l'unité naturelle (formulaires, scans).

4.7 Element / Structure

Un chunk COHÉRENT par section : le titre + TOUT ce qu'il contient (paragraphes, listes, code, tableaux [TABLE] inline, descriptions d'images) reste ensemble dans l'ordre de lecture. À l'inverse de Section/Heading, le tableau reste DANS le chunk de sa section.

Outil : Logique interne.

Paramètre	Défaut	Plage	Explication
Max size	2500	128-6000	Volontairement plus grand (les sections portent des tableaux) ; au-delà, re-découpe de secours.

Quand l'utiliser : rapports et manuels où le texte commente les tableaux/figures : un seul hit livre le tout.

4.8 Semantic

Embarque chaque phrase avec un modèle d'embedding MULTILINGUE puis coupe là où la similarité CHUTE (changement de sujet). S'exécute À L'INTÉRIEUR des sections : titres, tableaux et images restent protégés. Nettoyage final : fragments < 150c fusionnés, segments > 2x max re-découpés.

Outil : LlamaIndex SemanticSplitterNodeParser + sentence-transformers paraphrase-multilingual-MiniLM-L12-v2 (français inclus). Plus lent : chaque phrase est embarquée à l'indexation.

Paramètre	Défaut	Plage	Explication
Breakpoint percentile	75	50-95	Sensibilité de la coupe. HAUT (90-95) = seuls les grands changements de sujet coupent (peu de gros chunks). BAS (50-60) = coupe aux petites variations (beaucoup de petits chunks).
Max size	1200	128-6000	Plafond anti-emballlement pour les segments sans frontière de phrase.

Quand l'utiliser : longs textes SANS structure (pas de titres) mélangeant plusieurs sujets.

4.9 Hierarchical (parent-enfant)

Produit DEUX niveaux liés : des petits chunks ENFANTS (précis, utilisés pour la recherche) rattachés à de grands chunks PARENTS (le contexte, remis au LLM à la réponse). Chaque enfant porte parent_index. À coupler avec « Full Section » dans la config Retrieval.

Outil : Logique interne + LlamaIndex SentenceSplitter.

Paramètre	Défaut	Plage	Explication
Child size	400	64-4000	Taille des enfants = granularité de la recherche.
Parent = child x	4	2-8	Taille du parent = enfant x N. x4 avec des enfants de 400 donne des parents d'environ 1600 caractères.
Overlap	50	0-1000	Chevauchement entre chunks.

Quand l'utiliser : RAG avancé : précision de recherche ET contexte riche en même temps.

4.10 LLM-based (OpenAI) — prédiction de frontières

Le modèle NE TOUCHE JAMAIS au texte. Le moteur numérote les paragraphes d'une section ; le LLM renvoie UNIQUEMENT le regroupement ({"chunks": [[0,1],[2,3]]}) ; le moteur valide les ids (partition consécutive complète, sinon repli), reconstruit chaque chunk MOT POUR MOT depuis l'original, puis ajoute lui-même l'overlap. Garde-fous de coût : les sections < 1000 caractères ne consomment AUCUN appel GPT ; les grandes sections sont fenêtrées (9000c max par appel) pour ne jamais dépasser le contexte.

Outil : OpenAI (clé de l'espace -> fournisseur entreprise -> clé serveur) + moteur interne. Sans clé : repli silencieux sur Recursive (vérifier le badge de stratégie).

Paramètre	Défaut	Plage	Explication
Target size	1200	128-6000	Taille visée par chunk. Une INDICATION, pas une coupe dure : le modèle préfère terminer une idée complète plutôt que respecter le chiffre exactement.
Overlap	50	0-1000	Ajouté PAR LE MOTEUR après le choix des frontières (jamais par le modèle) : queue du chunk précédent, coupée à un mot.

Quand l'utiliser : sujets mélangés, texte narratif sans titres — quand la qualité prime sur le coût.

4.11 Agentic — 7 agents IA coopérants

L'IA ne choisit pas seulement OÙ couper : elle choisit COMMENT chunker. 1) Document Intelligence : profil riche (type, langue, densité tableaux/code/narratif, qualité des titres, complexité) + STRATÉGIE recommandée avec un score de confiance. 2) Strategy Agent : retient la stratégie (heading / element / semantic / llm / recursive) — confiance $\geq 0,6$ sinon règles déterministes. 3) Boundary Agent : frontières validées par ids (cf. 4.10). 4) Chunk Builder : assemblage structurel. 5) Chunk Reviewer : fusionne les miettes, re-découpe l'excès, répare les coupes en pleine phrase. 6) Metadata Agent : métadonnées STRUCTURÉES par chunk (titre, résumé, mots-clés, entités) stockées dans chunk_meta — jamais mélangées au texte, mais intégrées au vecteur d'embedding. 7) Chunk Evaluator : note chaque chunk 0-100 (taille, propreté des frontières, unité de sujet, métadonnées) ; sous 70, retour au Reviewer (boucle de feedback). Honnêteté : sans accès OpenAI, les chunks sont marqués agentic_fallback.

Outil : OpenAI (gpt) + toutes les stratégies ci-dessus. Coût borné : 1 appel d'analyse, frontières seulement sur les grandes sections, métadonnées par lots plafonnés.

Paramètre	Défaut	Plage	Explication
Granularity	balanced		

fine/balanced/c

oarse

Biais sur la taille planifiée : Fine = ~30% plus petit (précision),
Coarse = ~40% plus grand (contexte), Balanced = la
recommandation de l'analyse.

Target size	0 = auto	0-2500	0 = AUTO : l'agent d'analyse choisit la taille par document (doc dense -> plus petit, manuel structuré -> plus grand). Toute autre valeur fixe la taille.
Generate metadata	activé	on/off	Titre + résumé + mots-clés + entités par chunk, écrits par l'IA. Améliore la recherche ; coûte quelques appels API supplémentaires.

Quand l'utiliser : corpus hétérogènes : « choisis pour moi » avec traçabilité complète (stratégie choisie affichée, score Q).

5. Après le chunking — embedding et indexation

- Chaque chunk reçoit ses métadonnées : section §, hash md5, tokens (tiktoken), langue.
- Le texte embarqué = (titre + mots-clés agentic, si présents) + contenu ; le contenu stocké reste propre.
- Embedding à la dimension NATIVE du modèle de l'espace — plus de troncature à 1024.
- Stockage : texte + métadonnées dans PostgreSQL (chunks) ; vecteur dans chunk_vectors_<dim> (index HNSW par dimension, halfvec au-delà de 2000 dims).
- Changer de modèle d'embedding change la dimension : la plateforme exige la ré-indexation (drapeau automatique).

6. Récapitulatif des outils

Paramètre	Défaut	Plage	Explication
Docling	-	-	Parsing PDF : analyse de mise en page, titres, tableaux fusionnés multi-pages, images.
LlamaIndex	-	-	SentenceSplitter / TokenTextSplitter / SemanticSplitterNodeParser : toutes les découpes de texte.
sentence-transformers	-	-	Modèle multilingue paraphrase-multilingual-MiniLM-L12-v2 pour la coupe sémantique.
OpenAI (gpt)	-	-	Stratégie LLM (frontières par ids) et pipeline agentic (7 agents) ; descriptions d'images (vision).
tiktoken	-	-	Comptage exact des tokens par chunk.
PostgreSQL + pgvector	-	-	Stockage texte/métadonnées + vecteurs par buckets de dimension (HNSW, halfvec > 2000).
Moteur interne	-	-	Règles structurelles (tableaux entiers, § breadcrumbs), stratégies structurelles, overlap moteur, validations.